

Owen Muchini (a1877011) Assignment Part 2 Report

OpenMP N body simulation

The assignment goal was to investigate the following parts of parallel programming: keeping shared updates correct and distributing work across threads efficiently. In this case the fastest measured configuration was Reduced Forces Cyclic with four threads, however the Basic achieved the most speedup relative to its one thread runtime.

Setting up the comparisons

Before changing the parallel code, I wanted a consistent way to build the programs and check their results. Since there was and would be many implementations, which have different output modes, I wanted a consistent way to compile everything together and not accidentally use different settings. So I started with a Makefile so I could build all the versions using the same compiler options. To actually validate the parallel programs, I'd need to factor in how outputs could round the values and hide the smaller differences. So, I added a validation output mode that printed the exact precise values. I also implemented a python comparator to check them against base reference outputs. This helped in making it easier to test all of my changes before moving on to the next step.

Keeping the shared updates correct

For Part 2A, calculating each pair once meant adding its force to both particles. Different pairs could still involve the same particle, so two threads could overwrite each other's updates, like with the pairs (0, 2), (1,2) which would both update particle 2. I first used a critical section around the four force additions. I kept the distance and force calculations outside it so threads could still do that work at the same time.

Then I used one lock per particle to avoid making unrelated updates wait for the same critical section. Each lock protects both force components. To prevent a circular wait, I release one particle's lock before taking the other. I made sure to initialise the locks before the parallel region and destroy them afterwards.

I kept a parallel region around the timesteps, so that the same team could continue working. I also used default(none) to state which variables were shared or private. In each timestep, I first clear the old forces, then calculate the new forces before updating the particles' positions and velocities. The barrier at the end of each loop makes every thread wait until that work is finished to prevent a thread from moving a particle while the other is calculating based on previous positions.

I checked one-particle, uneven-size and non-collinear cases, repeating runs at the different threads (1 through to 32). The comparator is built to reject missing or non-finite states and checks masses exactly. Positions and velocities allow 0.000001 plus 0.0000000001 times the reference magnitude for rounding differences. Part 2A passed 156 comparisons and two standalone checks. Part 2B passed all 312 comparisons

Deciding where cyclic scheduling helps

For Part 2B, I compared the four variants asked for. Basic calculates interactions separately for each particle. The reduced versions calculate each pair once, keeps a force array per thread then merges those arrays. Reduced Default doesn't have any specific schedule clauses. For Forces Cyclic, `schedule(static,1)` applies only to the pair-force loop. Some threads could end up with more work if you simply divide equally based on the number of consecutive iterations. But with `schedule(static, 1)`, iterations are assigned to threads in turn, so every thread gets a mix of heavy and light calculations.

I also tested All Cyclic to see if using cyclic scheduling for all four loops would help. The other loops already divide the work more evenly, so there wasn't much to gain. At four threads, All Cyclic took 1.343 seconds compared with 1.244 seconds for Forces Cyclic, so applying it to every loop didn't help in this case.

Choosing and timing the workload

At the very beginning I ran some commands to check the machine attributes of the container. It was a Linux AMD64 container image on top of my Mac, running with Ubuntu 22.04.5, GCC 11.4.0 and libgomp. It had eight processors, which I checked and determined was not specific physical cores. I made sure to keep the build options consistent: `-O2 -g -Wall -Wextra -fopenmp`, with the maths library linked. I ran the container on my MacBook M1 Pro, which has eight CPU cores (6 performance and 2 efficiency) and 16 GB of memory.

Before collecting the final results, I tried 400 particles, but some runs finished in about 0.09 seconds. I was concerned that small delays from background activity would have an influence on the short measurements. So I increased the workload to 1600 particles while keeping 1000 timesteps. The calibration runs took about 1.2 to 11.7 seconds, giving me longer timings to compare. At a timestep of 0.01, one reduced result failed validation. I changed the timestep to 0.001 without relaxing the tolerance. With these all 20 thread combinations then passed.

To avoid collecting every result manually, I made a benchmark script for five runs at 1, 4, 8, 16 and 32 threads. It disabled dynamic team adjustment, checked team sizes and ran programs one at a time after compilation. I used bash scripts to help me test and perform benchmarks which help significantly, I was able to use knowledge from a previous course I did (Systems Programming) and some research. The benchmark script produced 100 measurements in a fixed order. I used the internal `omp_get_wtime` wall-clock timer so the results covered the parallel computation, barriers and team overhead, without having allocation and input generation. Below is the table of the mean runtimes.

Table 1. Mean wall-clock runtime in seconds from five runs.

Variant	1 Thread	4 Threads	8 Threads	16 Threads	32 Threads
Basic	11.639	3.105	2.919	2.775	2.718
Reduced Default	4.607	2.134	1.416	1.600	1.691
Forces Cyclic	4.570	1.244	1.428	1.537	1.656
All Cyclic	4.580	1.343	1.644	1.566	1.644

I then used a Python analysis script that I made to calculate the means, speedup and efficiency. Speedup divides each variant's one-thread mean by its mean at p threads; efficiency divides that speedup by p and multiplies by 100. Keeping each variant's own baseline let me compare how well it used more threads.

Results and interpretation

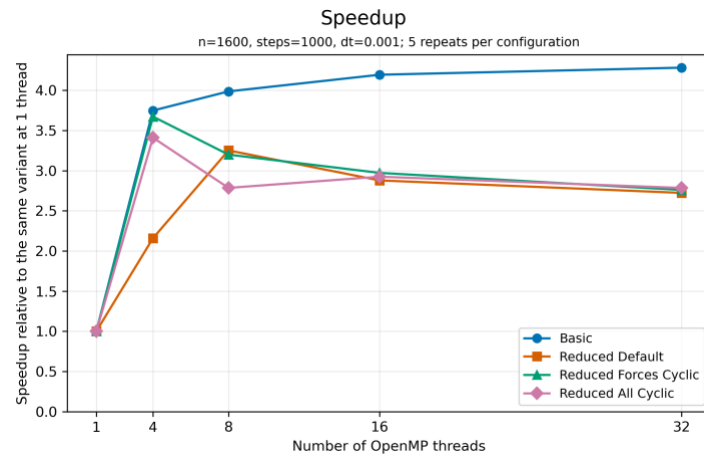


Figure 1. Speedup relative to each variant's own one-thread mean.

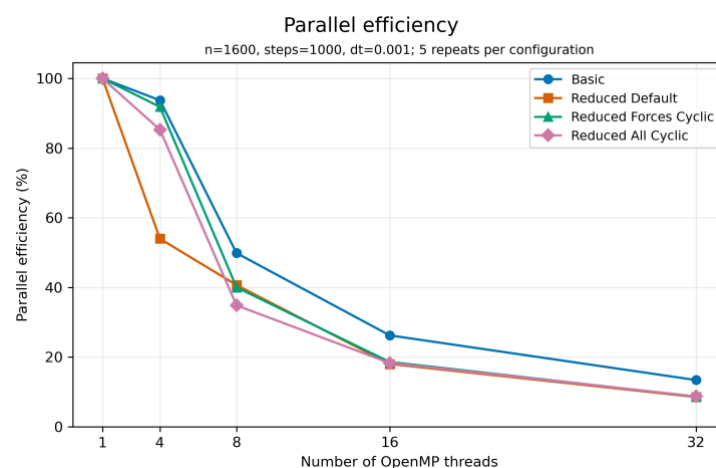


Figure 2. Efficiency falls as the thread count increases.

I used Python to plot the results so the trends were easier to compare. Forces Cyclic with four threads was fastest at 1.244 seconds, with a speedup of 3.673 and 91.83% efficiency. Basic achieved a higher speedup of 4.283 at 32 threads, but still took 2.718 seconds, showing why speedup alone does not identify the fastest version.

Adding threads didn't consistently help the reduced versions, possible due to array merging, waiting at barriers and running 32 threads on 8 processors. Applying scheduling to all loops didn't help much since All Cyclic was also slower than Forces Cyclic at 4 threads.

For this workload, I would choose Forces Cyclic with 4 threads. The results taught me to think about how work is divided and check actual runtimes, instead of simply assuming more threads will be faster.